

LECTURE - 10

Monday
28/9/26.

□ Randomised Quick Sort: i starts from start; j from end.

★ All of previous sorting algo. were "Comparison Based" algorithms. This causes extra cost of comparison i.e: $n \log n$.
So to get better/lower cost we need to eliminate comparison.

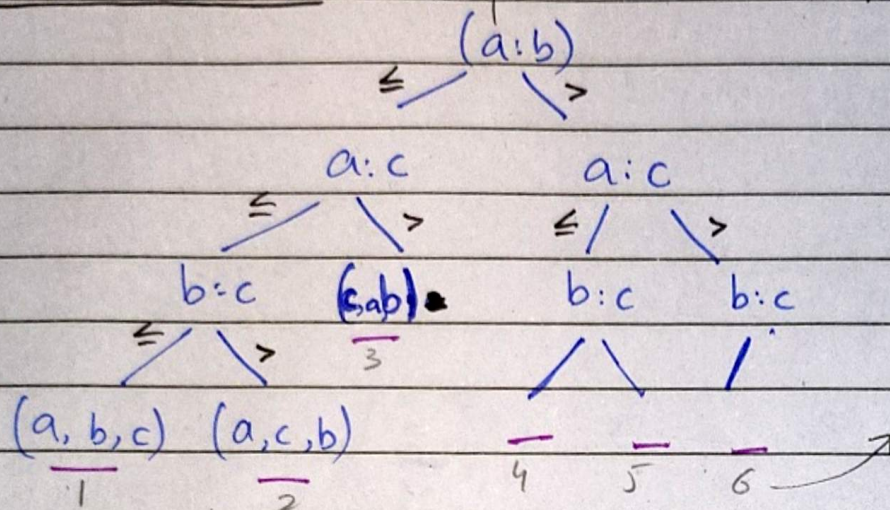
~~to some extent~~

- ★ Extra cost of comparison is proved by "Decision tree" (next page)
- ★ A better/lower cost solution could be 'Non-Comparison Based' Algo such as "Counting Sort".

Date: _____

data $\rightarrow (a, b, c)$

ordering



(no. of possible combin.)

$n! \leq \text{leaf nodes} < 2^h \rightarrow \text{height of tree}$

★ For finding which branch we need to choose we need to choose a branch which will have the height 'h' (for min. comparison)

$(n!) \leq 2^h$ (we need 'h' so that's why)

$$(n!) \leq 2^n$$

(we need 'h' so that's why)

* Take log on both sides

$$\log(n!) = \log 2^h \rightarrow h = \log(n!)$$

★ $\log(h!) = n \log n$ (proved by scientist)

$h = n \log n$ $\rightarrow$ Comparison can't be less than this.

1) Counting Sort: Count no. of appearances of each element in an array (i.e. freq).

- ★ We'll make an array ^{'meow' named} of size: k where ' k ' is the largest data value in the original array of size ' n '.
- ★ We'll then fill the k size ^(meow) array with all zeros.
- ★ Then whenever any data value comes in original array; suppose index 1 has data: 5; we'll go to 5th index of new k array and increment the data (i.e. count in it).
- ★ After all this; have an ' i ' pointer in original array's start and ' j ' pointer in start of k array.
- ★ Keep moving from ' j ' and if $meow[j]$ is not zero; then we'll put that ' j ' value in the original array's ' i ' index. (index of meow is the data value of original array).
- ★ Then we'll decrement $meow[j]$. If it's zero; move forward; else if $meow[j]$ is not zero; keep putting j in $Array[i]$ and keep incrementing ' i ' in original array.

Original Arr:

1	2	3	4	5	6	7	8	9
7	4	5	2	5	11	6	7	9

meow

1	2	3	4	5	6	7	8	9	10	11
0	1	0	1	2	1	2	0	1	0	1

↑ j

Code:

- Find largest element for 'k' $O(n)$
- Create array of k element initialised with all zeros $O(k)$
- for $i \leftarrow 1$ to n $O(n)$
 for $j \leftarrow 1$ to k $O(k)$
 while $meow[j] > 0$
 $A[j] = i$
 $\vdots$

$$T(n) = O(n+k)$$

* This is because the nested 'while' loop is only gonna work if the $meow[j] > 0$ which eliminates a lot of indexes. Therefore whichever is greater in 'n' or 'k'; that is $T(n)$



⇒ Stable Counting Sort: Repeated elements will again & again appear on the same index every time like if we've 5 & 5 then first 5 will always be at suppose 3rd index & second 5 always at 4th element.

→ Extra stuff to be added for stability:

- An extra array which will store cumulative sum of

	1	2	3	4	5	6	7	8	9			
Aw	5	4	7	6	11	9	7	2	5			
Original	0	0	1	0	1	2	1	2	0	1	0	1

1 appeared zero in original

2 appeared once so $1+0=1$

11 appeared once so $8+1=9$

Date _____

Meow
1:k

	1	2	3	4	5	6	7	8	9	10	11
	0	1	1	2	4	5	7	7	8	8	9

Sorted
1:n

1	2	3	4	5	6	7	8	9
2	4	5 _a	5 _b	6	7 _a	7 _b	9	11

⇒ Method for "Meow" (Cum. Array): Make meow array of size 'k' (largest data)

* we'll see the index of meow and see how many times that index appeared as data in the original array. That'll be COUNT

* Then add that COUNT to previous COUNT and put it in the meow array under that index.

* e.g: '5' appeared twice in original array therefore current count = 2.
And we'll add it in prev. count that is meow[4] which is '2'.
Therefore $2+2=4$. Now ~~do~~ meow[5] = 4.

⇒ Time Complexity of Stable Counting Sort: It'll be almost same as only increment will be $O(n+2k)$ which is still $O(n+k)$.